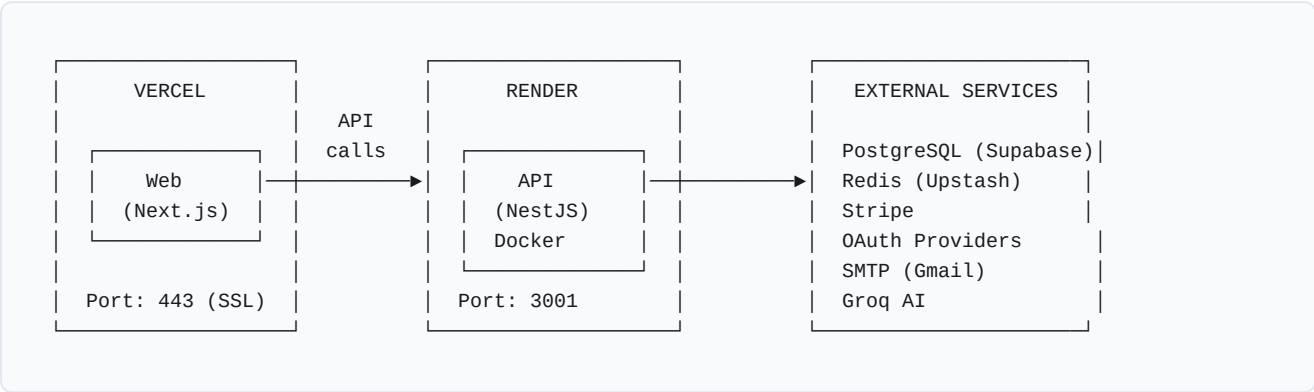


ATS Resume Builder — Deployment Guide

Project: novaai0401-ui/ats-resume-builder | Date: March 2026 | Version: 0.1.0



Monorepo Structure

Package	Framework	Deploys To	Purpose
resume-builder-web	Next.js 16 (App Router)	Vercel	Frontend web application
resume-builder-api	NestJS + Prisma	Render	Backend REST API
resume-builder-shared	TypeScript + Zod	Build dependency	Shared types & schemas
packages/resume-schemas	TypeScript + Zod	Build dependency	Resume validation schemas
packages/resume-intelligence	TypeScript	Build dependency	Resume parsing & mapping
resume-builder-mobile	React Native	App Store / Play Store	Mobile app (future)

1 External Services (Set Up FIRST)

These must be ready before deploying either the API or the Web app.

1A. PostgreSQL Database — Supabase

- Go to **supabase.com** → New Project
- Choose region close to Render (Oregon = US West)
- Set a strong database password
- Go to **Settings** → **Database**
- Copy **Connection string (URI)** → DATABASE_URL
- Copy **Direct connection** string → DIRECT_URL

Format:
DATABASE_URL = postgresql://postgres:[PASS]@db.[REF].supabase.co:6543/postgres
DIRECT_URL = postgresql://postgres:[PASS]@db.[REF].supabase.co:5432/postgres

1B. Redis — Upstash

- Go to **upstash.com** → Create Database → region US West
- Copy **UPSTASH_REDIS_REST_URL** → REDIS_URL
- Copy **UPSTASH_REDIS_REST_TOKEN** → REDIS_TOKEN

1C. Stripe (Billing)

1. Go to **dashboard.stripe.com** → Developers → API Keys
2. Copy Secret Key → `STRIPE_SECRET_KEY`
3. Create Webhook (after API deploy) → `STRIPE_WEBHOOK_SECRET`
4. Create two Price objects → `STRIPE_PRICE_STUDENT` , `STRIPE_PRICE_PRO`

1D. OAuth Providers (Social Login)

Provider	Console URL	Secrets You Get
Google	console.cloud.google.com → Credentials	<code>GOOGLE_LOGIN_CLIENT_ID</code> , <code>GOOGLE_LOGIN_CLIENT_SECRET</code>
GitHub	github.com/settings/developers → OAuth Apps	<code>GITHUB_CLIENT_ID</code> , <code>GITHUB_CLIENT_SECRET</code>
LinkedIn	linkedin.com/developers → My Apps	<code>LINKEDIN_CLIENT_ID</code> , <code>LINKEDIN_CLIENT_SECRET</code>
Yahoo	developer.yahoo.com → My Apps	<code>YAHOO_CLIENT_ID</code> , <code>YAHOO_CLIENT_SECRET</code>

Important: Set redirect URIs AFTER you know your Render API URL (Step 2).

1E. SMTP — Gmail with App Password

1. Enable 2FA on your Google account
2. Go to **myaccount.google.com/apppasswords** → Generate password

Variable	Value
<code>SMTP_HOST</code>	<code>smtp.gmail.com</code>
<code>SMTP_PORT</code>	<code>587</code>
<code>SMTP_SECURE</code>	<code>false</code>
<code>SMTP_USER</code>	<code>your-email@gmail.com</code>
<code>SMTP_PASS</code>	<code>your-16-char-app-password</code>
<code>SMTP_FROM</code>	<code>ATS Resume Builder <your-email@gmail.com></code>

1F. AI Service — Groq

1. Go to **console.groq.com** → API Keys → Create key
2. Copy → `GROQ_API_KEY`

1G. Generate Secrets (Run Locally)

```
# JWT_SECRET (64 hex chars)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# JWT_REFRESH_SECRET (64 hex chars)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# TOKEN_ENC_KEY (64 hex chars = 256-bit key)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

2 Deploy API on Render

2A. Create Render Service

- Go to **render.com** → Dashboard → **New** → **Web Service**
- Connect GitHub repo: `novaaio401-ui/ats-resume-builder`
- Configure:
 - Name:** `ats-resume-builder-api`
 - Region:** Oregon
 - Branch:** `main`
 - Root Directory:** *(leave empty — uses repo root)*
 - Runtime:** Docker
 - Dockerfile Path:** `./Dockerfile`
 - Plan:** Starter (\$7/month) or higher

2B. Set ALL Environment Variables in Render Dashboard

Go to your Render service → **Environment** tab → add each variable:

Category	Variable	Value
Core	NODE_ENV	production
	PORT	3001
Database	DATABASE_URL	From Step 1A (pooled)
	DIRECT_URL	From Step 1A (direct)
Auth	JWT_SECRET	From Step 1G
	JWT_REFRESH_SECRET	From Step 1G
	TOKEN_ENC_KEY	From Step 1G
CORS	CORS_ORIGIN	<code>https://your-web.vercel.app</code> (update after Step 3)
Redis	REDIS_URL	From Step 1B
	REDIS_TOKEN	From Step 1B
Stripe	STRIPE_SECRET_KEY	From Step 1C
	STRIPE_WEBHOOK_SECRET	From Step 1C
	STRIPE_PRICE_STUDENT	From Step 1C
	STRIPE_PRICE_PRO	From Step 1C
	STRIPE_SUCCESS_URL	<code>https://your-web.vercel.app/dashboard</code>
	STRIPE_CANCEL_URL	<code>https://your-web.vercel.app/dashboard</code>
Google OAuth	GOOGLE_LOGIN_CLIENT_ID	From Step 1D
	GOOGLE_LOGIN_CLIENT_SECRET	From Step 1D
	GOOGLE_LOGIN_REDIRECT_URI	<code>https://your-api.onrender.com/auth/social/google/callback</code>
	SOCIAL_LOGIN_SUCCESS_URL	<code>https://your-web.vercel.app/auth/callback</code>

GitHub OAuth	GITHUB_CLIENT_ID	From Step 1D
	GITHUB_CLIENT_SECRET	From Step 1D
	GITHUB_REDIRECT_URI	<code>https://your-api.onrender.com/auth/social/github/callback</code>
LinkedIn OAuth	LINKEDIN_CLIENT_ID	From Step 1D
	LINKEDIN_CLIENT_SECRET	From Step 1D
	LINKEDIN_REDIRECT_URI	<code>https://your-api.onrender.com/auth/social/linkedin/callback</code>
SMTP	SMTP_HOST	From Step 1E
	SMTP_PORT	From Step 1E
	SMTP_SECURE	<code>false</code>
	SMTP_USER	From Step 1E
	SMTP_PASS	From Step 1E
	SMTP_FROM	From Step 1E
AI	AI_PROVIDER	<code>groq</code>
	GROQ_API_KEY	From Step 1F
	GROQ_MODEL	<code>llama-3.3-70b-versatile</code>
Admin	ADMIN_EMAILS	<code>novaai0401@gmail.com</code>

2C. Run Database Migrations

After Render deploys, run from your local machine:

```
cd resume-builder-api
DATABASE_URL="your-direct-connection-url" npx prisma migrate deploy
```

2D. Verify API

Visit `https://your-api.onrender.com/health` — should return a success response.

Note your Render API URL — you need it for the next steps.

3 Deploy Web on Vercel

3A. Create Vercel Project

1. Go to **vercel.com** → **Add New Project**
2. Import repo: `novaai0401-ui/ats-resume-builder`
3. Configure:
 - **Framework:** Next.js (auto-detected)
 - **Root Directory:** `resume-builder-web`
 - **Build & Install Commands:** leave default (reads from `vercel.json`)

3B. Set Environment Variables in Vercel Dashboard

Before clicking Deploy, go to **Settings** → **Environment Variables**:

Variable	Value	Environments
<code>NEXT_PUBLIC_API_URL</code>	<code>https://ats-resume-builder-api.onrender.com</code>	Production, Preview, Development
<code>NEXT_PUBLIC_SESSION_IDLE_TIMEOUT_MS</code>	<code>1800000</code>	All
<code>NEXT_PUBLIC_ADMIN_EMAILS</code>	<code>novaai0401@gmail.com</code>	All

3C. Deploy

Click **Deploy** — Vercel builds and deploys automatically.

Note your Vercel URL (e.g. `https://ats-resume-builder.vercel.app`)

4 Cross-Link URLs (Critical!)

Now that both services are live, update the cross-references.

4A. Update Render (API) Env Vars

Variable	Update To
<code>CORS_ORIGIN</code>	<code>https://ats-resume-builder.vercel.app</code>
<code>SOCIAL_LOGIN_SUCCESS_URL</code>	<code>https://ats-resume-builder.vercel.app/auth/callback</code>
<code>STRIPE_SUCCESS_URL</code>	<code>https://ats-resume-builder.vercel.app/dashboard</code>
<code>STRIPE_CANCEL_URL</code>	<code>https://ats-resume-builder.vercel.app/dashboard</code>
<code>GOOGLE_OAUTH_SUCCESS_REDIRECT</code>	<code>https://ats-resume-builder.vercel.app/dashboard</code>

4B. Update OAuth Provider Redirect URIs

Go back to each OAuth console and set:

Provider	Redirect URI
Google	<code>https://ats-resume-builder-api.onrender.com/auth/social/google/callback</code>
GitHub	<code>https://ats-resume-builder-api.onrender.com/auth/social/github/callback</code>

LinkedIn	https://ats-resume-builder-api.onrender.com/auth/social/linkedin/callback
Yahoo	https://ats-resume-builder-api.onrender.com/auth/social/yahoo/callback

4C. Update Stripe Webhook

In Stripe Dashboard → Webhooks → set endpoint URL:

<https://ats-resume-builder-api.onrender.com/billing/webhook>

5 Set Up GitHub Actions (CI/CD)

5A. Get Vercel Credentials

```
# Install Vercel CLI
npm install -g vercel

# Login
vercel login

# Link web project
cd resume-builder-web && vercel link
# → .vercel/project.json contains orgId + projectId

# Get your API token
# vercel.com → Settings → Tokens → Create
```

5B. Add GitHub Secrets

Repo **Settings** → **Secrets and variables** → **Actions** → **New repository secret**:

Secret Name	Where to Get It
VERCEL_TOKEN	vercel.com → Settings → Tokens → Create
VERCEL_ORG_ID	.vercel/project.json → orgId
VERCEL_WEB_PROJECT_ID	.vercel/project.json → projectId
NEXT_PUBLIC_API_URL	Your Render API URL

5C. Create GitHub Environment

Repo Settings → Environments → New environment → name: `production`

6 Verify Everything

Test	URL	Expected Result
API Health	https://your-api.onrender.com/health	200 OK
Web Home	https://your-web.vercel.app	Landing page renders
Login Page	https://your-web.vercel.app/auth/login	Social buttons + email/password form
Social Login	Click "Continue with Google"	Redirects to Google → back to dashboard
Register	Fill register form	Account created → dashboard
Create Resume	Dashboard → New resume	Resume editor loads
Export PDF	Click Export PDF in editor	PDF downloads

Deployment Sequence — Summary

PHASE 1 – External Services

① Supabase (PostgreSQL) → DATABASE_URL, DIRECT_URL

- ② Upstash (Redis) → REDIS_URL, REDIS_TOKEN
- ③ Groq (AI) → GROQ_API_KEY
- ④ Gmail App Password (SMTP) → SMTP_* variables
- ⑤ Generate JWT + encryption secrets locally

PHASE 2 – Deploy API (Render)

- ⑥ Create Render Web Service (Docker)
- ⑦ Set all environment variables
- ⑧ Wait for build + deploy
- ⑨ Run: `prisma migrate deploy`
- ⑩ Verify: `GET /health` → 200 OK
- ⑪ Note your API URL

PHASE 3 – Deploy Web (Vercel)

- ⑫ Create Vercel project (root: `resume-builder-web`)
- ⑬ Set `NEXT_PUBLIC_API_URL` = Render API URL
- ⑭ Deploy
- ⑮ Note your Web URL

PHASE 4 – Cross-Link

- ⑯ Set `CORS_ORIGIN` on Render → Vercel Web URL
- ⑰ Update OAuth redirect URIs → Render API URL
- ⑱ Update Stripe webhook → Render API URL
- ⑲ Update success/cancel URLs → Vercel Web URL

PHASE 5 – CI/CD

- ⑳ Add GitHub Secrets (`VERCEL_TOKEN`, `ORG_ID`, `PROJECT_ID`)
- ㉑ Create "production" environment

PHASE 6 – Verify

- ㉒ Full end-to-end test: `register` → `login` → `resume` → PDF

Key Rule: External services FIRST → API SECOND → Web THIRD → Cross-link LAST.